

Funciones y Algoritmos

Clase 4 · Taller de Programación en Python

Lucas Gómez Tobón

Tópicos

01

Funciones

Crear herramientas
reutilizables

02

Algoritmos

Pensar paso a paso
para resolver problemas

03

Recursión*

Funciones que se
llaman a sí mismas

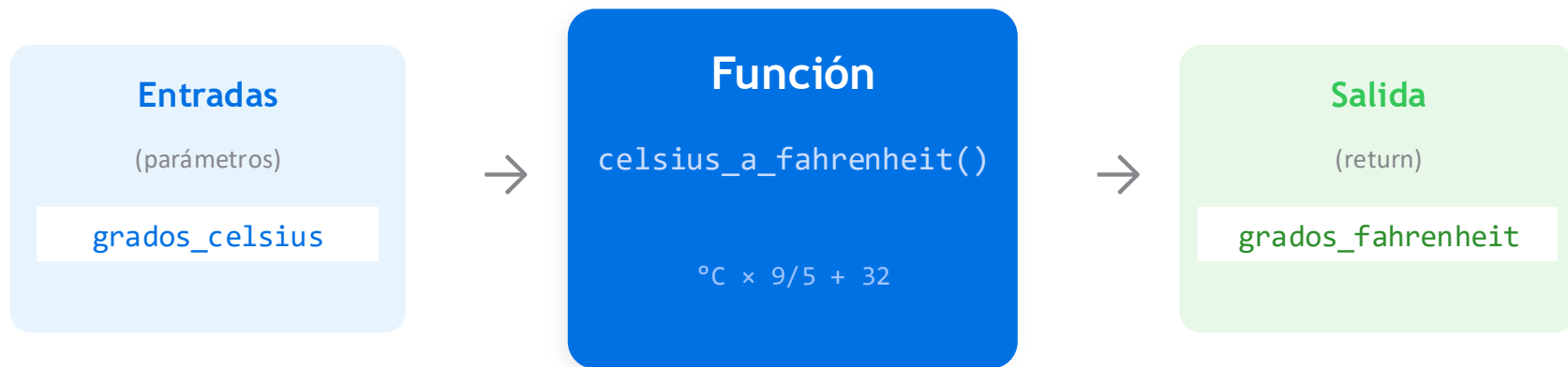
Los algoritmos son ideas, las funciones son la herramienta para implementarlas.

Parte 1

Funciones

Crear nuestras propias herramientas

¿Qué es una función?



¿Por qué crear funciones?

Reutilizar código · Reducir errores · Código más legible · Combinar piezas simples

Anatomía de una función

```
def celsius_a_fahrenheit(celsius):  
    """Convierte Celsius a Fahrenheit."""  
    resultado = celsius * (9/5) + 32  
    return resultado
```



def

Palabra clave para definir



nombre

Descriptivo, snake_case



docstring

Documentación de la función



cuerpo

Las instrucciones que se ejecutan



return

Valor que entrega al terminar

return vs print

Una de las confusiones más comunes al comenzar

print()

Le habla al humano

- Muestra texto en la consola
El dato se pierde tras mostrarse
No se puede usar después

```
resultado = doblar(5) # None
```

return

Le habla al programa

- Entrega un valor al código
Puede guardarse en variable
Se puede seguir operando

```
resultado = doblar(5) # 10 ✓
```

Parámetros

Obligatorios

Deben pasarse siempre al llamar la función

```
def f(capital, tiempo):
```

Con valor por defecto

Opcionales — se usa el default si se omiten

```
def f(capital, tasa=0.05):
```

Retornar múltiples valores

Python empaqueta varios returns en una tupla que puede desempaquetarse

```
min, max, prom = estadisticas_basicas(notas)
```

Composición de funciones

Construir soluciones complejas combinando piezas simples

`esta_aprobado()`

`contar_aprobados()`

`estadisticas_basicas()`



`reporte_curso(lista_notas)`

*Cada función hace una sola cosa bien.
La función principal las ensambla.*

Parte 2

Algoritmos

Pensar antes de codificar

¿Qué es un algoritmo?

Una secuencia finita, ordenada y precisa de pasos para resolver un problema.



Problema

Lo que queremos resolver

"Necesito el valor máximo de una lista"



Algoritmo

La estrategia para resolverlo

"Recorro actualizando el ganador"



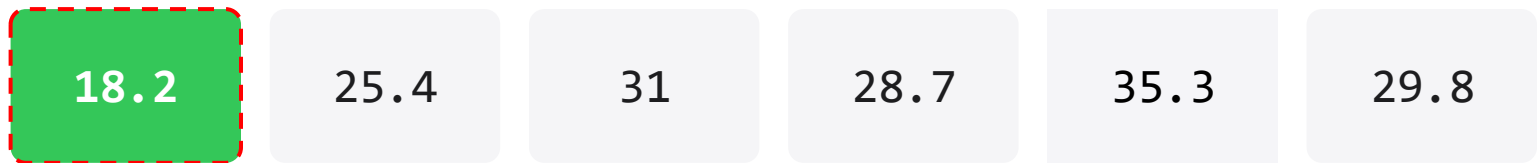
Función

La implementación en Python

```
def encontrar_maximo(lista):
```

Algoritmo: Encontrar el máximo

Lista de temperaturas:



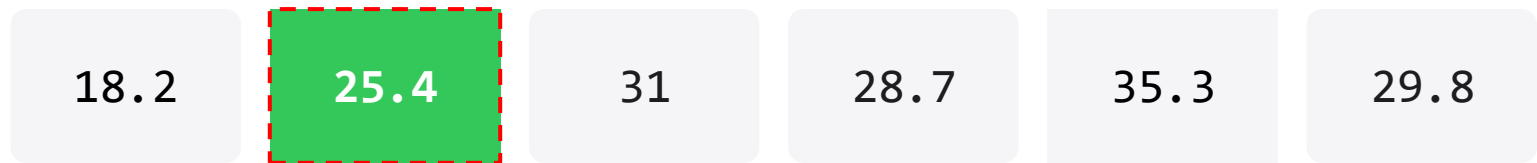
Paso a paso:

Inicio

ganador = 18.2

Algoritmo: Encontrar el máximo

Lista de temperaturas:



Paso a paso:

Inicio ganador = 18.2

Paso 2 $25.4 > 18.2 \rightarrow \text{ganador} = 25.4$

Algoritmo: Encontrar el máximo

Lista de temperaturas:

18.2

25.4

31

28.7

35.3

29.8

Paso a paso:

Inicio ganador = 18.2

Paso 2 25.4 > 18.2 → ganador = 25.4

Paso 3 31.0 > 25.4 → ganador = 31.0

Algoritmo: Encontrar el máximo

Lista de temperaturas:



Paso a paso:

Inicio	ganador = 18.2
Paso 2	25.4 > 18.2 → ganador = 25.4
Paso 3	31.0 > 25.4 → ganador = 31.0
Paso 4	28.7 < 31.0 → sin cambio

Algoritmo: Encontrar el máximo

Lista de temperaturas:

18.2

25.4

31

28.7

35.3

29.8

Paso a paso:

Inicio

ganador = 18.2

Paso 2

25.4 > 18.2 → ganador = 25.4

Paso 3

31.0 > 25.4 → ganador = 31.0

Paso 4

28.7 < 31.0 → sin cambio

Paso 5

35.3 > 31.0 → ganador = 35.3

Algoritmo: Encontrar el máximo

Lista de temperaturas:

18.2

25.4

31

28.7

35.3

29.8

Máximo

Paso a paso:

Inicio	ganador = 18.2
Paso 2	25.4 > 18.2 → ganador = 25.4
Paso 3	31.0 > 25.4 → ganador = 31.0
Paso 4	28.7 < 31.0 → sin cambio
Paso 5	35.3 > 31.0 → ganador = 35.3
Paso 6	29.8 < 35.3 → sin cambio

Resultado

35.3

°C más alta

El proceso de 3 pasos

1

Palabras

Describir en lenguaje natural, como si le explicarás a un amigo.

2

Pseudocódigo

Notación estructurada, a medio camino entre el español y Python.

3

Código Python

Traducir el pseudocódigo a una función. Se traduce casi directamente.

Pseudocódigo

```
funcion encontrar_maximo(lista):  
    ganador ← lista[0]  
  
    para cada elemento en lista:  
        si elemento > ganador:  
            ganador ← elemento  
  
    retornar ganador
```

Algoritmo: Búsqueda Lineal

¿Está este elemento en la lista? ¿En qué posición?

Ana

[0]

Carlos

[1]

María

[2]

Juan

[3]

Laura

[4]

¡Encontrado!

```
funcion busqueda_lineal(lista, obj):  
    para i desde 0 hasta len-1:  
        si lista[i] == obj:  
            retornar i  
    retornar -1 # no encontrado
```

Puntos clave

- Recorre de inicio a fin
- Se detiene al encontrar
- Retorna -1 si no existe
- Peor caso: revisa todo

Algoritmo: Bubble Sort

I'm trying to learn the **bubble sort algorithm** in Python. Could you create a **step-by-step visual explanation** that allows the user to input the list to sort? Please set the default list to [5, 3, 8, 1, 4].



Bubble Sort → Aplicación: Mediana

Pseudocódigo

```
funcion bubble_sort(lista):  
    n ← longitud de lista  
  
    repetir n veces (pasada i):  
        para j desde 0 a n-i-2:  
            si lista[j] > lista[j+1]:  
                intercambiar  
    retornar lista
```

¿Para qué ordenar?

- Calcular la mediana
- Calcular percentiles
- Construir rankings
- Eliminar duplicados

Mediana: valor central de datos ordenados

Ejemplo: Ingresos mensuales (miles)

Ordenados: [1500, 2100, 2600, 2800, 3200, 3900, 4100, 4500, 6800, 12000]

Promedio: 4,350 (inflado por 12,000) Mediana: 3,550 (más representativa)

Parte 3

Recursión

Funciones que se llaman a sí mismas

Los 2 ingredientes de la recursión



Caso Base

La condición de parada.
El caso más simple que se resuelve directamente.

```
if n == 0: return 1
```



Paso Recursivo

La llamada a sí misma con una versión más pequeña del problema.

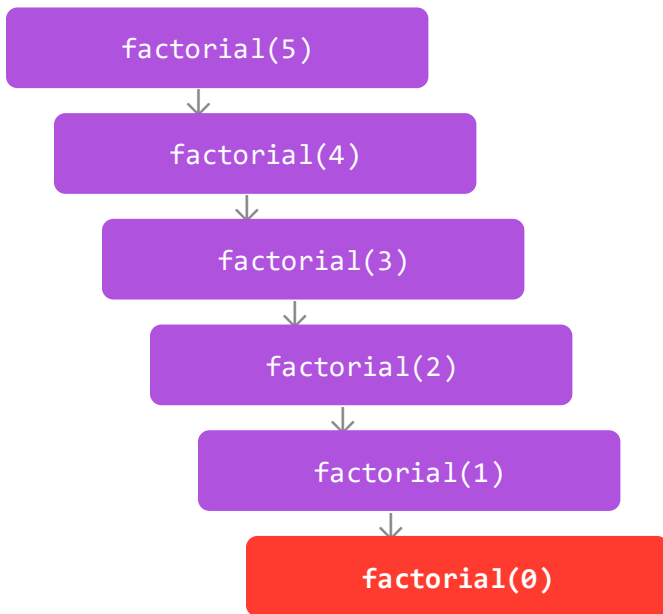
```
return n * factorial(n-1)
```

⚠ Sin caso base → recursión infinita → el programa nunca termina

Factorial: La pila de llamadas

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Se apilan →



← Se resuelven

`factorial(5) → 5 × 24 = 120`

`factorial(4) → 4 × 6 = 24`

`factorial(3) → 3 × 2 = 6`

`factorial(2) → 2 × 1 = 2`

`factorial(1) → 1 × 1 = 1`

`factorial(0) → 1`

Fibonacci

Cada número es la suma de los dos anteriores



casos base

$$\text{fib}(0) = 0$$

$$\text{fib}(1) = 1$$

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$

Dato curioso: la razón áurea

La proporción entre términos consecutivos converge a 1.618...
Este número aparece en la naturaleza, el arte y la arquitectura.

$$\varphi = 1.618...$$

Recursión vs Iteración

Iteración (for/while)

Recursión

Legibilidad

Mejor para procesos
lineales paso a paso

Mejor cuando el problema
se define en sí mismo

Memoria

Eficiente: una sola
pila de ejecución

Mayor: múltiples
llamadas pendientes

Velocidad

Generalmente
más rápida

Puede ser más lenta
por sobrecarga

Casos de uso

Recorrer listas,
acumular resultados

Árboles, fórmulas
matemáticas inductivas

Regla práctica: si el problema es naturalmente recursivo, usa recursión. Para recorrer listas, usa bucles.

Resumen: el proceso



Entender

el problema en
palabras simples

Búsqueda de máximo

Un recorrido, actualizando el
ganador

Diseñar

el algoritmo en
pseudocódigo

Búsqueda lineal

Recorrer uno a uno hasta
encontrar

Implementar

como una función
en Python

Bubble Sort

Comparar pares adyacentes e
intercambiar

Verificar

con ejemplos
conocidos

Recursión

Caso base + llamada más
pequeña